CASM 생성 과정

개요

CASM(Cairo Assembly) 생성은 Cairo 컴파일 파이프라인의 마지막 단계로, Sierra 중간표현을 실행 가능한 CASM 코드로 변환하는 과정입니다. 이 단계는 `cairo-lang-sierra-to-casm` crate에서 구현되며, Sierra 명령어를 Cairo VM에서 실행할 수 있는 저수준 어셈블리 명령어로 매핑합니다.

cairo-lang-sierra-to-casm Crate 구조

핵심 모듈 구성

```
cairo-lang-sierra-to-casm/
├── src/
| ├── lib.rs # 라이브러리 진입점
| ├── compiler.rs # 메인 컴파일러 로직
| ├── invocations/ # Sierra 명령어별 CASM 변환
| ├── environment/ # 컴파일 환경 관리
| ├── references.rs # 메모리 참조 관리
| ├── relocations.rs # 코드 재배치 처리
| ├── annotations.rs # 프로그램 주석 처리
| ├── metadata.rs # 메타데이터 관리
| └── circuit.rs # 회로 정보 처리
```

주요 데이터 구조

CairoProgram

```
pub struct CairoProgram {
pub instructions: Vec<Instruction>,
pub debug_info: CairoProgramDebugInfo,
pub consts_info: ConstsInfo,
}
```

CASM 프로그램의 최종 표현으로, 실행 가능한 명령어 목록과 디버그 정보, 상수 정보를 포함합니다.

CompiledInvocation

```
pub struct CompiledInvocation {
pub instructions: Vec<Instruction>,
pub relocations: Vec<RelocationEntry>,
pub results: Vec<BranchChanges>,
pub environment: Environment,
}
```

개별 Sierra 명령어의 컴파일 결과를 나타내며, 생성된 CASM 명령어와 재배치 정보, 분기 변경사항을 포함합니다.

Sierra 명령어의 CASM 매핑 규칙

1. 기본 산술 연산

Felt252 연산 예시

```
// Sierra: felt252_add(a, b) -> (result)

// CASM: [ap] = [fp - 3] + [fp - 4]; ap++
```

Sierra의 felt252 덧셈은 CASM의 덧셈 명령어로 직접 매핑됩니다:

```
fn build_felt252_op_with_var(

builder: CompiledInvocationBuilder<'_>,

op: Felt252BinaryOperator,

) -> Result<CompiledInvocation, InvocationError> {

let [a, b] = builder.try_get_single_cells()?;

let mut casm_builder = CasmBuilder::default();

let cell_op = match op {

Felt252BinaryOperator::Add => CellOperator::Add,

Felt252BinaryOperator::Sub => CellOperator::Sub,

Felt252BinaryOperator::Mul => CellOperator::Mul,

Felt252BinaryOperator::Div => CellOperator::Div,

};

let res_var = casm_builder.bin_op(cell_op, a, b);

// ...

}
```

2. 메모리 연산

store_temp 구현

```
fn build_store_temp(
builder: CompiledInvocationBuilder<'_>,
ty: &ConcreteTypeId,
) -> Result<CompiledInvocation, InvocationError> {
let expression = builder.try_get_refs::<1>()?[0];
let type_size = builder.program_info.type_sizes[ty];
// AP 레지스터를 사용하여 임시 값 저장
let instructions = get_store_instructions(
&builder,
ty,
repeat_n(CellRef { register: Register::AP, offset: 0 }, type_size as usize),
expression,
)?;
}
```

3. 제어 흐름

분기 명령어

Sierra의 분기는 CASM의 조건부 점프로 변환됩니다:

```
// Sierra: branch_align() -> ()
// CASM: (분기 정렬을 위한 NOP 또는 점프)
```

```
// Sierra: jump(target) -> never

// CASM: jmp rel target; (상대 점프)
```

4. 함수 호출

```
// Sierra: function_call(func_id, args...) -> (results...)

// CASM: call rel func_offset; (함수 호출)
```

함수 호출은 스택 프레임 설정과 함께 CASM call 명령어로 변환됩니다.

레지스터 할당 알고리즘

레지스터 모델

Cairo VM은 세 가지 주요 레지스터를 사용합니다:

1. **AP (Allocation Pointer)**: 스택의 현재 위치
2. **FP (Frame Pointer)**: 현재 함수 프레임의 시작
3. **PC (Program Counter)**: 현재 실행 중인 명령어 위치

참조 표현 (ReferenceExpression)

```
pub struct ReferenceExpression {
pub cells: Vec<CellExpression>,
}
```

```
pub enum CellExpression {
Deref(CellRef), // [reg + offset]
DoubleDeref(CellRef, i16), // [[reg + offset] + offset2]
Immediate(BigInt), // 즉시값
BinOp { op: CellOperator, a: Var, b: Var }, // 이항 연산
}
```

스택 관리

```
pub struct ReferenceValue {
pub expression: ReferenceExpression,
pub ty: ConcreteTypeId,
pub stack_idx: Option<usize>, // 연속 스택에서의 인덱스
pub introduction_point: IntroductionPoint,
}
```

각 값은 스택에서의 위치와 도입 지점을 추적하여 효율적인 메모리 접근을 보장합니다.

AP 추적 시스템

```
pub enum ApTracking {
Disabled,
Enabled {
ap_change: usize, // 기준점으로부터의 AP 변화량
```

```
base: ApTrackingBase, // AP 추적 기준점
},
}
```

AP 추적을 통해 컴파일러는 각 지점에서의 스택 상태를 정확히 파악할 수 있습니다.

메모리 레이아웃 최적화

1. 스택 최적화

```
fn validate_stack_top(
ap_change: ApChange,
branch_signature: &BranchSignature,
refs: &[OutputReferenceValue],
) {
// 스택 상단의 변수들이 연속적인지 검증
let stack_top_vars = UnorderedHashMap::<usize, usize>::from_iter(
branch_signature.vars.iter().enumerate().filter_map(|(arg_idx, var)| {
if let OutputVarReferenceInfo::NewTempVar { idx: stack_idx } = var.ref_info {
Some((stack_idx, arg_idx))
} else {
None
}
}),
```

```
);
// 연속성 검증 로직...
}
```

2. 프레임 상태 관리

```
pub enum FrameState {
BeforeAllocation, // 로컬 변수 할당 전
Allocating { allocated: usize }, // 할당 중
Finalized { allocated: usize }, // 할당 완료
}
```

함수 프레임의 상태를 추적하여 로컬 변수의 효율적인 할당을 관리합니다.

3. 상수 세그먼트 최적화

```
pub struct ConstsInfo {
pub segments: OrderedHashMap<u32, ConstSegment>,
pub total_segments_size: usize,
pub circuit_segments: OrderedHashMap<ConcreteTypeId, u32>,
}

pub struct ConstSegment {
pub values: Vec<BigInt>, // 세그먼트의 값들
```

```
pub const_offset: UnorderedHashMap<ConcreteTypeId, usize>, // 타입별 오프셋
pub segment_offset: usize, // 코드 세그먼트로부터의 오프셋
}
```

상수들은 별도 세그먼트로 관리되어 메모리 사용량을 최적화합니다.

CASM 코드 생성 과정 상세 분석

1. 컴파일 초기화

```
pub fn compile(
program: &Program,
program_info: &ProgramRegistryInfo,
metadata: &Metadata,
config: SierraToCasmConfig,
) -> Result<CairoProgram, Box<CompilationError>> {
let mut instructions = Vec::new();
let mut relocations: Vec<RelocationEntry> = Vec::new();
let mut sierra_statement_info: Vec<SierraStatementDebugInfo> = Vec::new();
// 메타데이터 검증
validate_metadata(program, &program_info.registry, metadata)?;
// 프로그램 주석 생성
let mut program_annotations = ProgramAnnotations::create(
program.statements.len(),
```

```
backwards_jump_indices,
&program.funcs,
metadata,
config.gas_usage_check,
&program_info.type_sizes,
)?;
}
```

2. 문장별 컴파일

각 Sierra 문장은 다음 과정을 거쳐 CASM으로 변환됩니다:

```
for (statement_id, statement) in  program.statements.iter().enumerate() {
let  statement_idx = StatementIdx(statement_id);
match  statement {
Statement::Return(ref_ids) => {
// 반환 문 처리
let  ret_instruction = RetInstruction {};
instructions.push(Instruction::new(InstructionBody::Ret(ret_instruction), false)
}
Statement::Invocation(invocation) => {
// 함수 호출 처리
let  compiled_invocation = compile_invocation(
program_info,
```

```
invocation,
libfunc,
statement_idx,
&invoke_refs,
annotations.environment,
)?;
instructions.extend(compiled_invocation.instructions);
relocations.extend(compiled_invocation.relocations);
}
}
}
```

3. 재배치 처리

```
pub  enum Relocation {
RelativeStatementId(StatementIdx), // 상대 문장 ID
SegmentStart(u32), // 세그먼트 시작
ConstStart(u32, ConcreteTypeId), // 상수 시작
CircuitStart(ConcreteTypeId), // 회로 시작
EndOfProgram, // 프로그램 끝
}

impl Relocation {
```

```
pub fn apply(
&self,
instruction_offset: CodeOffset,
statement_offsets: &[CodeOffset],
consts_info: &ConstsInfo,
instruction: &mut Instruction,
) {
let target_pc = match self {
Relocation::RelativeStatementId(statement_idx) =>
statement_offsets[statement_idx.0],
Relocation::SegmentStart(segment_index) => {
let segment = consts_info.segments.get(segment_index).unwrap();
*statement_offsets.last().unwrap() + segment.segment_offset
}
// 기타 재배치 타입들...
};
// 명령어에 재배치 적용
match instruction {
Instruction { body: InstructionBody::Call(CallInstruction {
target: DerefOrImmediate::Immediate(value), relative: true
}), .. } => {
value.value += target_pc as i128 - instruction_offset as i128;
}
```

```
// 기타 명령어 타입들...
}
}
}
```

4. 최종 프로그램 생성

```
// 상수 세그먼트 정보 생성
let  consts_info = ConstsInfo::new(
program_info,
program.libfunc_declarations.iter().map(|ld| &ld.id),
&circuits_info.circuits,
const_segments_max_size,
)?;

// 재배치 적용
relocate_instructions(&relocations, &statement_offsets, &consts_info, &mut  inst

// 최종 프로그램 반환
Ok(CairoProgram {
instructions,
consts_info,
debug_info: CairoProgramDebugInfo { sierra_statement_info },
```

```
})
```

환경 관리 시스템

컴파일 환경

```
pub struct Environment {
pub ap_tracking: ApTracking, // AP 추적 정보
pub stack_size: usize, // 스택 크기
pub frame_state: FrameState, // 프레임 상태
pub gas_wallet: GasWallet, // 가스 지갑
}
```

분기 변경사항

```
pub struct BranchChanges {
pub refs: Vec<OutputReferenceValue>, // 새로운 참조들
pub ap_change: ApChange, // AP 변화량
pub ap_tracking_change: ApTrackingChange, // AP 추적 변화
pub gas_change: CostTokenMap<i64>, // 가스 변화량
pub clear_old_stack: bool, // 기존 스택 정리 여부
pub new_stack_size: usize, // 새로운 스택 크기
```

```
}
```

최적화 기법

1. 명령어 수준 최적화

- **인라인 상수**: 작은 상수값들은 즉시값으로 인라인 처리
- **레지스터 재사용**: 동일한 값에 대한 중복 로드 제거
- **AP 증분 최적화**: 연속된 메모리 접근 시 AP 증분 최소화

2. 메모리 접근 최적화

- **스택 지역성**: 관련된 데이터를 스택에서 인접하게 배치
- **프레임 포인터 활용**: 로컬 변수 접근 시 FP 기반 주소 지정 사용
- **더블 디레퍼런스 최적화**: 포인터 체인 최적화

3. 분기 최적화

- **분기 예측**: 자주 사용되는 분기 경로 최적화
- **분기 정렬**: 분기 대상의 메모리 정렬 최적화
- **점프 테이블**: 다중 분기 시 점프 테이블 사용

디버그 정보 생성

Sierra 문장 디버그 정보

```
pub struct SierraStatementDebugInfo {
pub start_offset: usize, // 바이트코드에서의 시작 오프셋
```

```
pub end_offset: usize, // 바이트코드에서의 끝 오프셋
pub instruction_idx: usize, // 명령어 벡터에서의 인덱스
pub additional_kind_info: StatementKindDebugInfo, // 문장 종류별 추가 정보
}

pub enum StatementKindDebugInfo {
Return(ReturnStatementDebugInfo), // 반환 문 정보
Invoke(InvokeStatementDebugInfo), // 호출 문 정보
}
```

PC-Sierra 매핑

```
impl CairoProgram {
pub fn sierra_statement_index_by_pc(&self, pc: usize) -> StatementIdx {
StatementIdx(
self.debug_info.sierra_statement_info
.partition_point(|x| x.start_offset <= pc) - 1,
)
}
}
```

이를 통해 실행 시점의 PC 값으로부터 해당하는 Sierra 문장을 역추적할 수 있습니다.

에러 처리 및 검증

컴파일 에러 타입

```
pub  enum CompilationError {
ProgramRegistryError(Box<ProgramRegistryError>),
AnnotationError(AnnotationError),
InvocationError { statement_idx: StatementIdx, error: InvocationError },
ReturnArgumentsNotOnStack { statement_idx: StatementIdx },
ReferencesError { statement_idx: StatementIdx, error: ReferencesError },
LibfuncInvocationMismatch { statement_idx: StatementIdx },
DanglingReferences { statement_idx: StatementIdx, var_id: VarId },
ExpectedBranchAlign { source_statement_idx: StatementIdx, destination_statement_
// 기타 에러들...
}
```

검증 과정

1. **메타데이터 검증**: 가스 정보와 AP 변화 정보의 일관성 검사
2. **타입 검증**: 참조와 타입의 일치성 검사
3. **스택 검증**: 스택 상태의 일관성 검사
4. **분기 검증**: 분기 대상의 유효성 검사

성능 고려사항

컴파일 시간 최적화

- **병렬 처리**: 독립적인 문장들의 병렬 컴파일
- **캐싱**: 반복되는 패턴의 컴파일 결과 캐싱
- **지연 평가**: 필요한 시점에만 복잡한 계산 수행

생성 코드 품질

- **코드 크기**: 불필요한 명령어 제거로 코드 크기 최소화
- **실행 속도**: 빈번한 연산의 최적화된 명령어 시퀀스 생성
- **메모리 사용량**: 효율적인 스택 및 힙 사용 패턴

결론

CASM 생성 과정은 Sierra 중간표현을 실행 가능한 Cairo Assembly로 변환하는 복잡한 과정입니다. 이 과정에서는 효율적인 레지스터 할당, 메모리 레이아웃 최적화, 그리고 정확한 코드 재배치가 핵심적인 역할을 합니다. `cairo-lang-sierra-to-casm` crate는 이러한 모든 과정을 체계적으로 관리하여 고품질의 CASM 코드를 생성합니다.